Name : Ananya Shet

SRN : PES2UG23CS065

Section : A

Date : 17-02-2026

Unit 2

Part 1a: LangChain Setup & Models

LangChain is a framework that simplifies building applications using Large Language Models (LLMs). Without LangChain, we would need to manually adjust parameters and rewrite code for each experiment.

Environment Setup

1. Install LangChain and langchain-google-genai.
2. Generate a Google API key.

code sends a question - Google's cloud runs the model - The answer comes back - print it.

Temperature Parameter

We created two versions of the same model (Gemini 2.5 Flash) with different temperature settings. Temperature controls randomness in word prediction:

When Temperature is 0.0 : Highly deterministic, consistent output

When Temperature is 1.0 : More creative, varied, and less predictable output.

When temperature is 2.0 : Extremely random, often chaotic or nonsensical responses.

So we're not changing the model itself — we're changing **how it chooses the next word** during generation.

We run the same prompt twice for both models to compare.

When temperature is 0.0

7. Experiment: Consistency vs. Creativity

We will ask both models to "Define the word 'Idea' in one sentence." We will run the code **TWICE** for each model.

Hypothesis:

- The Focused model (Temp=0) should say the *exact same thing* both times.
- The Creative model (Temp=1) should say *different things*.

[4]

```
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")
```

```
--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or suggestion that is formed or exists in the mind.
Run 2: An idea is a thought, concept, or mental image formed in the mind.
```

[5]

```
print("--- CREATIVE (Temp=1) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

```
--- CREATIVE (Temp=1) ---
Run 1: An idea is a mental construct, concept, or plan formed in the mind, often representing a suggestion, understanding, or solution.
Run 2: An idea is a thought, concept, or plan conceived in the mind.
```

When temperature changed to 2.0

4. Prompt Templates: The Safe Way

Don't do THIS: `prompt = f"Translate {user_input} to Spanish"`

Do THIS: `ChatPromptTemplate`.

It handles messy input (like quotes or newlines) safely.

```
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ("system", "You are a translator. Translate {input_language} to {output_language}."),
    ("human", "{text}")
])

# We can check what inputs it expects
print(f"Required variables: {template.input_variables}")
```

```
Required variables: ['input_language', 'output_language', 'text']
```

7. Experiment: Consistency vs. Creativity

We will ask both models to "Define the word 'Idea' in one sentence." We will run the code **TWICE** for each model.

Hypothesis:

- The Focused model (Temp=0) should say the *exact same thing* both times.
- The Creative model (Temp=1) should say *different things*.

```
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")
```

```
--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, plan, or suggestion that originates in the mind.
Run 2: An idea is a mental concept, impression, or thought that can originate spontaneously or through contemplation.
```

```
print("--- CREATIVE (Temp=1) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

```
--- CREATIVE (Temp=1) ---
Run 1: An idea is a thought, concept, or mental impression that exists in the mind, often serving as a plan, suggestion, or starting point for action or understanding.
Run 2: An idea is a thought, concept, or mental impression that exists in the mind.
```

At temperature 0, the model is highly deterministic, always choosing the most probable words, which produces identical, precise outputs each time. At temperature 2, the model becomes very creative and

random, exploring less likely words and sentence structures, which leads to significant differences between runs. Even at high temperatures, the model avoids nonsense because its internal training biases favor coherent and meaningful text. Essentially, higher temperature increases diversity at the cost of consistency.

Partb: Prompts and Parsers

Pipeline : The input is filled into a structured template. The prompt is sent to the AI (Gemini/OpenAI), which returns an AIMessage object. Output Parser Extracts only the text you care about from the AIMessage.

String: Just plain text, no roles or context.

Message: Text with a role

Context windows : Maximum amount of text that model can remember

LCEL: LCEL (LangChain Expression Language) is a way to chain AI steps together cleanly using the pipe | operator. Combines **all steps into one reusable pipeline**. You just **call chain.invoke()** and get the final text.

Assignment

Create a chain that:

1. Takes a movie name.
2. Asks for its release year.
3. Calculates how many years ago that was (You can try just asking the LLM to do the math).

Try to do it in **one line of LCEL**.

```python
from langchain_google_genai import ChatGoogleGenerativeAI

# Initialize the Gemini LLM
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.0)

# Movie name
movie = "Inception"

# Prompt: ask Gemini to calculate years ago in one line
prompt = f"Movie: {movie}\nAsk for its release year and calculate how many years ago it was. Answer in one line."

# Get response
response = llm.invoke(prompt)
print(response.content)
```

```
Inception was released in 2010, which was 14 years ago.
```

We will give the movie name to LLM and find its year and calculates how many years ago it was released. The model returns the answer as an AIMessage, This is a single step approach.

Unit 2 - Part 2a: The Anatomy of a Prompt

Understanding Randomness

- AI predicts the next token based on probabilities.
- Temperature affects token selection → explains variation in answers.

Prompt Engineering (CO-STAR Framework)

Structure prompts with:

- **Context:** Who/what is acting
- **Objective:** Task to perform
- **Style:** Formal, funny, etc.
- **Tone:** Polite, direct, empathetic
- **Audience:** Who reads it
- **Response Format:** Text, JSON, list

Structured CO-STAR prompts produce more accurate and predictable outputs than lazy prompts.

Hallucination vs Creativity

- AI fills gaps with probable answers when facts are missing → normal behavior.
- Precise prompts → controlled, accurate output.

Unit 2 - Part 2b: Zero-Shot to Few-Shot

1. **In-Context Learning:** The model learns patterns from the text/examples in the prompt without retraining.
2. **Zero-Shot:** No examples given; the model relies on its pretrained knowledge.
3. **Few-Shot:** Provide examples; the model detects patterns and applies them to new inputs.

2. Zero-Shot (No Context)

The model relies purely on its training data.

```
prompt_zero = "Combine 'Angry' and 'Hungry' into a funny new word."
print(f"Zero-Shot: {llm.invoke(prompt_zero).content}")
```

```
Zero-Shot: The most common and widely accepted funny new word for combining 'Angry' and 'Hungry' is:

**Hangry**
```

3. Few-Shot (Pattern Matching)

We provide examples. The Attention Mechanism attends to the **Structure** (`Input -> Output`) and the **Tone** (Sarcasm).

```
prompt_few = """
Combine words into a funny new word. Give a sarcastic definition.

Input: Breakfast + Lunch
Output: Brunch (An excuse to drink alcohol before noon)

Input: Chill + Relax
Output: Chillax (What annoying people say when you are panic attacks)

Input: Angry + Hungry
Output:
"""
print(f"Few-Shot: {llm.invoke(prompt_few).content}")
```

```
Few-Shot: Output: Hangry (The scientifically recognized condition that allows you to be an absolute monster until someone hands you a snack.)
```

Unit 2 - Part 2c: Advanced Templates & Theory

Type	Modification	Cost	Use case
Hard Prompt	Input text only	Cheap	Rapid Prototyping
Soft Prompt	Model weights (fine-tuning)	Expensive	Domain-specific tasks

Hard Prompts (Prompt Engineering) - Large Language Model (GPT, Gemini, Claude, etc.)

Soft Prompts (Fine-Tuning) - Training frameworks (Hugging Face, OpenAI fine-tuning API)

Dynamic Few-Shotting:

- Use semantic search to find the most relevant examples.
- Select top-k similar examples.
- Inject only those into the prompt.

Few-Shot Container

Inserts selected examples into the prompt. Keeps formatting consistent. LangChain sends the structured prompt to the LLM.

3. Analysis :

- Clear Separation: Separates system instructions, examples, and user query for better structure.
- Structural Clarity: Model reads instructions first, examples next, and input last, reducing confusion.
- No Training Required: Model does not update weights; it learns patterns using in-context learning.

Unit 2 - Part 3a: Chain of Thought (CoT)

Part 3a: Chain of Thought (CoT)

- Forces the model to reason **step-by-step** before giving the final answer.
- Helps avoid shallow guessing and improves accuracy, especially for multi-step problems.
- **Example:** Solving (5 + 2) × 3

  Normal prompt → might compute only 5 + 2 = 7. CoT prompt → computes intermediate steps: 2 × 3 = 6 → 5 + 6 = 11

Unit 2 - Part 3b: Tree of Thoughts (ToT) & Graph of Thoughts (GoT)

- **Tree of Thoughts(ToT):** Explores multiple branches, uses a "Judge" to select optimal solution.
- **Graph of Thoughts (GoT):** Produces multiple drafts and synthesizes into a stronger final output.
- **LangChain RunnableParallel** enables parallel reasoning paths.

Conclusion: Linear (CoT), branching (ToT), and network-style (GoT) thinking enhance reasoning quality and creativity.

Parametric vs Non-Parametric Memory & RAG

Memory Types:

- **Parametric:** Stored in LLM weights; learned during training.
- **Non-Parametric:** Stored externally (e.g., vector databases like FAISS); retrieved at query time.

Vector DB vs Normal DB

- **Vector DB:** Stores embeddings for semantic similarity search.

- **Normal DB:** Stores raw structured data; exact match queries.

Retrieval-Augmented Generation (RAG)

- Purpose: Improves LLMs by retrieving external knowledge rather than relying solely on internal memory.

Working:
Convert text into embeddings (numeric vectors).
Store embeddings in a vector database (e.g., FAISS).
Measure similarity with cosine similarity (e.g., Cat vs Dog > Cat vs Car).
Retrieve relevant documents and inject into prompt before generating an answer.

Key Benefits:
Reduces hallucinations
Grounds responses in external facts
Combines parametric memory (LLM weights) + non-parametric memory (vector DB)
Makes answers more accurate, controllable, and updatable

Naive RAG Pipeline

- Works like an open-book exam:
    - External knowledge stored in a small KB → converted to embeddings → stored in FAISS.
    - User query → retriever finds closest vectors → context injected into prompt → LLM generates answer.
- Reduces hallucinations by ensuring answers rely on retrieved facts rather than only model memory.

Indexing Algorithms (FAISS)

Method	Description	Pros	Cons
Flat Index	Brute-force search over all vectors	100% accurate	Slow (O(N)), impractical for large datasets
IVF (Inverted File)	Cluster-based search	Faster	Slight accuracy loss
HNSW (Graph-Based Search)	Builds graph connections → navigates shortcuts	Extremely fast	High RAM usage
PQ (Product Quantization)	Compresses vectors to save memory	Memory-efficient	Slight accuracy loss; best for billions of vectors

Core Idea: FAISS balances speed, accuracy, and memory — you cannot maximize all three at once. Intelligent indexing enables large-scale vector search efficiently, forming the backbone of real-world RAG systems.

Conclusion:
RAG and vector DBs allow LLMs to ground outputs in relevant external knowledge, improving reliability, precision, and scalability. FAISS indexing strategies ensure fast, memory-efficient retrieval for large datasets.